TRANSFORMER ARCHITECTURE

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. Transformer Fundamentals

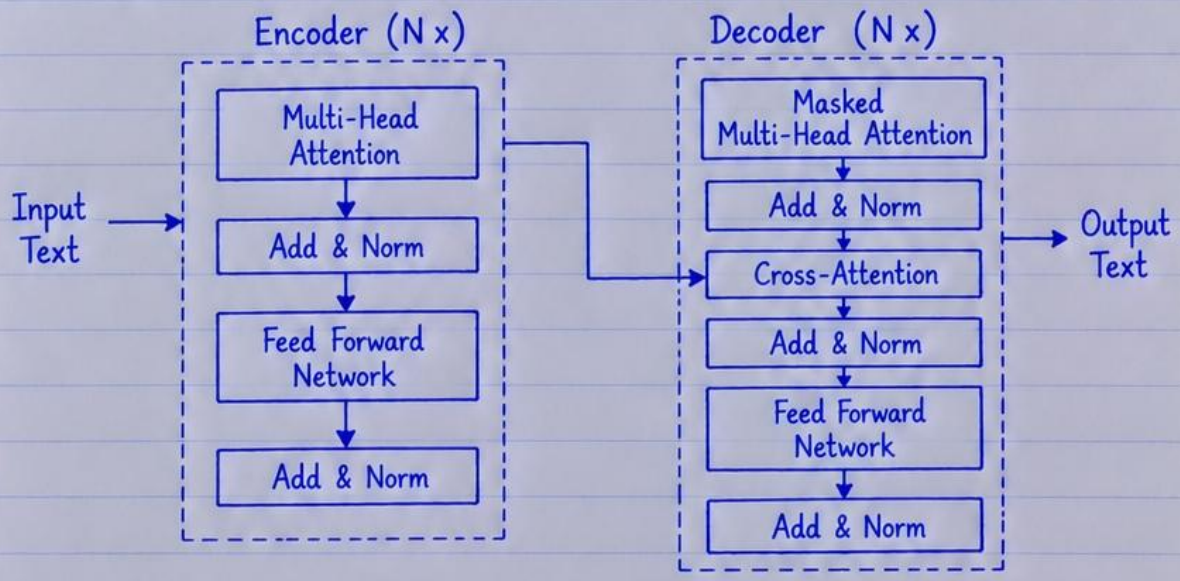
1. What is a Transformer ?

- Transformer is a deep learning model architecture introduced in 2017 in the paper "Attention Is All You Need" by Google.
- It is mainly used for natural language processing (NLP) but now widely used in computer vision, speech and multi-modal models.
- It relies on the self-attention mechanism instead of recurrence (RNN) or convolution (CNN).
- It can process all tokens in parallel, making it more efficient.

2. Key Characteristics

- Uses self-attention to capture long-range dependencies.
- No RNN or CNN, so fully parallelizable.
- Works well on large-scale data.
- Scales effectively with model size and data.
- Becomes the foundation of modern LLMs (GPT, BERT, T5 etc).
- Handles variable length input sequences.

3. High Level Architecture



4. Why Transformer is Important ?

- Handles long-range context better than RNN/CNN.
- Enables pre-training at massive scale.
- Powers modern LLMs like GPT, BERT, LLaMA, T5 etc.
- Used beyond NLP in vision (ViT), speech (Whisper) and multi-modal models.

Rahul Yadav

2. Tokens, Embeddings & Positional Encoding

1. Tokenization

- Tokenization converts input text into smaller units called tokens.
- A token can be a word, subword or even a character.
- Transformers usually use subword tokenization (e.g. BPE, WordPiece) to handle rare words and unknown words.
- Each token is then mapped to a unique integer id using a vocabulary.

Example:

Text : I love machine learning

↓

Tokens : ["I", "love", "machine", "learning"]

↓

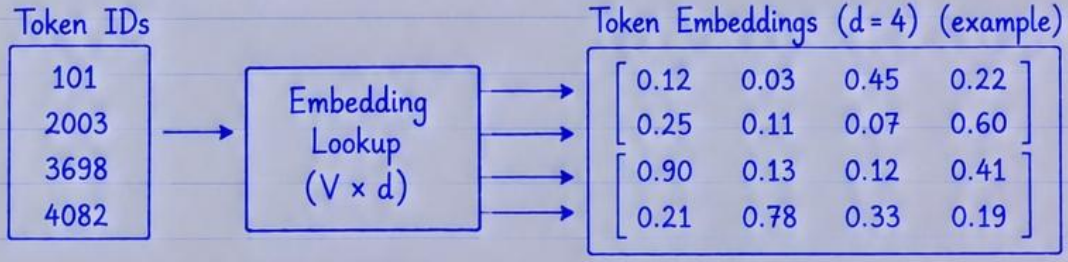
Token IDs : [101, 2003, 3698, 4082]

Vocabulary (sample)

I	→	101
love	→	2003
machine	→	3698
learning	→	4082
...	→	...

2. Token Embeddings

- Each token id is converted into a dense vector of fixed size (embedding).
- These vectors are learned during training.
- Embeddings capture the semantic meaning of tokens.
- If vocabulary size is V and embedding dimension is d, then embedding matrix has shape (V × d).



3. Positional Encoding

- Transformers do not have recurrence, so they do not know the position of tokens in a sequence.
- Positional encoding adds position information to token embeddings.
- It can be fixed (sinusoidal) or learnable.
- It is added element-wise to the token embeddings.

Example (Sinusoidal Positional Encoding)

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Final Input to Transformer

Input Embedding = Token Embedding + Positional Encoding

Token Embedding					Positional Encoding					Final Input				
0.12	0.03	0.45	0.22		0.01	0.02	0.01	0.03		0.13	0.05	0.46	0.25	
0.25	0.11	0.07	0.60	+	0.04	0.01	0.02	0.01	=	0.29	0.12	0.09	0.61	
0.90	0.13	0.12	0.41		0.03	0.02	0.01	0.04		0.93	0.15	0.13	0.45	
0.21	0.78	0.33	0.19		0.01	0.03	0.02	0.01		0.22	0.81	0.35	0.20	

Rahul Yadav

3. Self-Attention Intuition

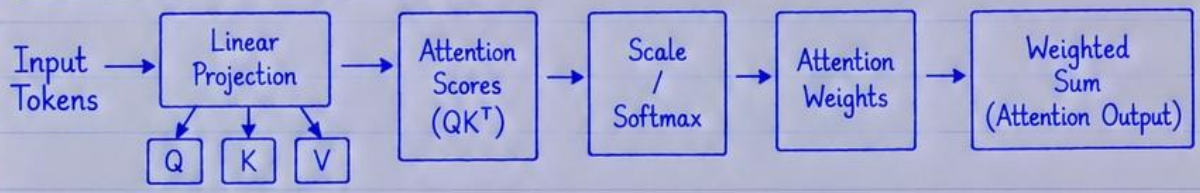
1. What is Self-Attention ?

- Self-attention allows each token to attend to all other tokens in the input sequence.
- It helps the model understand the context and relationships between words, regardless of their distance.
- It computes attention scores to find how much each token should focus on other tokens.

2. Core Idea (Intuition)

- For each token, we create three vectors:
  - Query (Q) → what we are looking for
  - Key (K) → what we can match with
  - Value (V) → the actual information to retrieve
- We compute similarity between query and keys to get attention scores.
- Use softmax to convert scores into probabilities.
- Use these probabilities to get a weighted sum of values.

3. Self-Attention Flow



4. Mathematical Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q : Query matrix $(n \times d_k)$
- K : Key matrix $(n \times d_k)$
- V : Value matrix $(n \times d_v)$
- d_k : dimension of keys/queries
- n : number of tokens (sequence length)

- QK^T → similarity scores
- Divide by $\sqrt{d_k}$ → scale to avoid large values
- Softmax → convert to probabilities
- Multiply with V → get final output

5. Example (Small Example)

Input tokens: [I , love , AI]

Assume $d_k = 2$

Query Matrix (Q)

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{matrix} (I) \\ (love) \\ (AI) \end{matrix}$$

Key Matrix (K)

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{matrix} (I) \\ (love) \\ (AI) \end{matrix}$$

Attention Scores (QK^T)

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Softmax (row-wise)

$$\begin{bmatrix} 0.42 & 0.16 & 0.42 \\ 0.16 & 0.68 & 0.16 \\ 0.33 & 0.33 & 0.33 \end{bmatrix}$$

→ Multiply with V → Output

Rahul Yadav

4. Queries, Keys & Values (Q, K, V)

1. What are Q, K and V ?

- Each input token is converted into three vectors: Query (Q), Key (K) and Value (V) using linear transformations.
- Q is used to find which tokens to attend to.
- K is used to compute similarity with the query,
- V contains the actual information to be aggregated.
- These are learned during training.

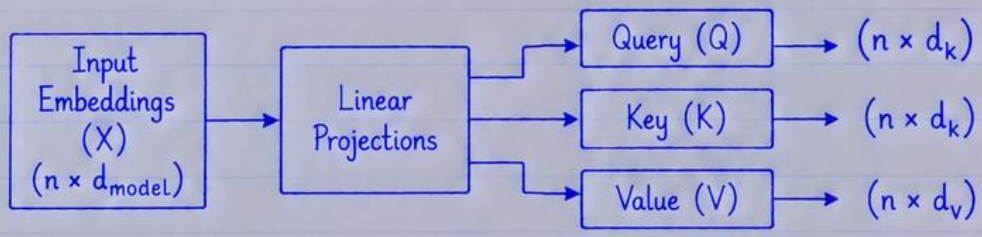
2. How are Q, K and V Generated ?

- Each token embedding (x) is multiplied with three learnable weight matrices.
- W_Q, W_K and W_V are trained parameters.
- This is a linear projection, so it's just a matrix multiplication.

$$Q = XW_Q \qquad K = XW_K \qquad V = XW_V$$

X : Input embeddings $(n \times d_{model})$
W_Q : $(d_{model} \times d_k)$
W_K : $(d_{model} \times d_k)$
W_V : $(d_{model} \times d_v)$
Q, K : $(n \times d_k)$
V : $(n \times d_v)$

3. Visual Representation



4. Example with Numbers

Let's take a simple example with 3 tokens and $d_{model} = 4$, $d_k = d_v = 2$

Input Embeddings (X) (3 × 4)

$$\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}$$

W_Q (4 × 2)

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

W_K (4 × 2)

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

W_V (4 × 2)

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$$

- Each token now has a Query, Key and Value vector.
- Q and K are used to compute attention scores (similarity).
- V is used to get the final context (weighted sum).

$Q = XW_Q$ (3 × 2)

$$\begin{bmatrix} 2 & 1 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$K = XW_K$ (3 × 2)

$$\begin{bmatrix} 2 & 1 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$V = XW_V$ (3 × 2)

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$$

Rahul Yadav

5. Scaled Dot-Product Attention

1. What is Scaled Dot-Product Attention ?

- It is the core attention mechanism used in Transformers.
- It computes the attention scores between all tokens using Queries (Q) and Keys (K)
- The scores are scaled, passed through a softmax to get attention weights.
- These weights are then multiplied with Values (V) to get the final output.
- It allows the model to focus on the most relevant tokens in the input sequence.

2. Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q : Query matrix $(n \times d_k)$
- K : Key matrix $(n \times d_k)$
- V : Value matrix $(n \times d_v)$
- d_k : dimension of keys/queries
- n : number of tokens
- QK^T : similarity scores
- Softmax : convert to probabilities
- Multiply with V : get final output

3. Step by Step Process

Input Tokens Embeddings (X) → Generate Q, K, V (linear projection) → Compute Attention Scores (QK^T) → Scale ($\div \sqrt{d_k}$) → Apply Softmax (Attention weights) → Multiply with V (final output)

4. Numerical Example (Small)

Let's take 3 tokens with $d_k = d_v = 2$

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \quad V = \begin{bmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \end{bmatrix}$$

Step 1: Compute QK^T

$$QK^T = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

Step 2: Scale ($d_k = 2$, $\sqrt{d_k} = 1.41$)

$$\text{Scaled} = \frac{1}{1.41} \times \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 0.71 & 0 & 0.71 \\ 0 & 0.71 & 0.71 \\ 0.71 & 0.71 & 1.41 \end{bmatrix}$$

Step 3: Softmax (row-wise)

$$\text{Attention Weights} \approx \begin{bmatrix} 0.40 & 0.20 & 0.40 \\ 0.20 & 0.40 & 0.40 \\ 0.25 & 0.25 & 0.50 \end{bmatrix}$$

- Each row sums to 1
- These are the attention weights

Step 4: Multiply with V

$$\text{Output} = \begin{bmatrix} 1.20 & 0.60 \\ 0.60 & 1.20 \\ 1.00 & 1.00 \end{bmatrix}$$

- Each token is now a weighted sum of all value vectors.
- The model can focus on relevant tokens.

5. Key Takeaways

- Scaled dot-product attention is simple but very powerful.
- Scaling by $\sqrt{d_k}$ prevents large values and stabilizes training.
- Softmax converts scores into attention weights.
- The final output is a weighted sum of value vectors.

Rahul Yadav

6. Multi-Head Attention

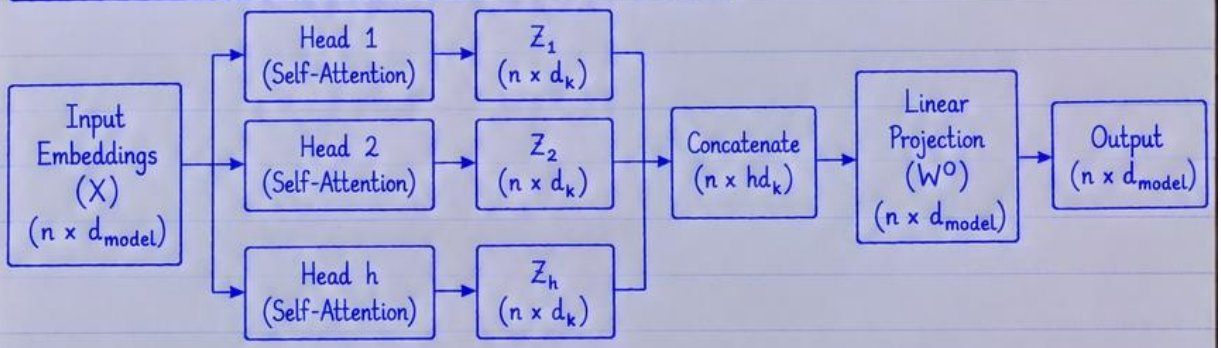
1. What is Multi-Head Attention ?

- Multi-Head Attention (MHA) runs multiple self-attention heads in parallel.
- Each head learns different patterns and focuses on different parts of the input sequence.
- It allows the model to capture diverse relationships and better understand the context.
- The outputs of all heads are concatenated and projected to get the final output.

2. Why Multiple Heads ?

- Different heads can attend to different tokens/positions.
- Some heads may capture local relationships (nearby words).
- Some heads may capture long-range dependencies.
- It gives the model a richer and more powerful representation.

3. Architecture of Multi-Head Attention



4. Mathematical Formulation

- For each head i (1 to h):
  1) $Q_i = XW_i^Q$
  2) $K_i = XW_i^K$
  3) $V_i = XW_i^V$
  4) $head_i = Attention(Q_i, K_i, V_i)$

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- X : Input embeddings $(n \times d_{model})$
- W_i^Q, W_i^K, W_i^V : Learnable weight matrices $(d_{model} \times d_k)$
- h : Number of heads
- d_k : Dimension per head (usually d_{model}/h)
- Concatenate all heads : $Z = [head_1, head_2, \ldots, head_h]$
- Final output : $Output = ZW^O$ $(n \times d_{model})$

5. Example Dimensions (BERT Base)

- $d_{model} = 768$
- Number of heads (h) = 12
- $d_k = d_v = d_{model}/h = \frac{768}{12} = 64$
- Each head output : $(n \times 64)$
- Concatenated output : $(n \times 12 \times 64) = (n \times 768)$
- After linear projection : $(n \times 768)$

6. Key Takeaways

- Multi-Head Attention runs multiple attention heads in parallel.
- Each head learns different relationships.
- Helps capture both local and global context.
- Concatenation + linear projection gives the final output.
- Widely used in all modern Transformer models like BERT, GPT, T5, etc.

Rahul Yadav

7. Positional Encoding

1. Why Positional Encoding ?

- Transformers do not have recurrence or convolution.
- Self-attention is position invariant (it only sees the content, not the order).
- Without positional information, the model cannot distinguish between different word orders.
- Positional encoding adds information about the position of each token in the sequence.

2. Types of Positional Encoding

- Fixed (Sinusoidal) Positional Encoding → Used in original Transformer (Vaswani et al.).
- Learnable Positional Encoding → Position embeddings learned during training.
- Relative Positional Encoding → Uses relative distance between tokens (e.g. in T5, RoPE, etc.).

3. Sinusoidal Positional Encoding (Original Transformer)

- Uses sine and cosine functions of different frequencies.
- Each position has a unique positional vector.
- Works well and can generalize to longer sequences.
- Formula :

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

pos : position of token $(0,1,2, \ldots n-1)$
i : dimension index $(0,1,2, \ldots, d_{model}/-1)$
d_{model} : embedding dimension
$PE(pos, 2i)$ → even index (sin)
$PE(pos, 2i+1)$ → odd index (cos)

4. Example (Small)

Let $d_{model} = 4$ (so $i = 0, 1$) and position = 0, 1, 2

Position	PE(0)	PE(1)	PE(2)	PE(3)
0	0.00	1.00	0.00	1.00
1	0.84	0.54	0.01	1.00
2	0.91	-0.42	0.02	1.00

- Each row is the positional vector for that position.
- Even dimensions use sine.
- Odd dimensions use cosine.
- Values change smoothly with position.

5. Adding Positional Encoding to Token Embeddings

- Final Input to Transformer = Token Embedding + Positional Encoding
- Both have the same dimension (d_{model}).
- This helps the model understand the order of tokens in the sequence.

Token Embedding $(n \times d_{model})$ + Positional Encoding $(n \times d_{model})$ → Input to Transformer $(n \times d_{model})$

6. Key Takeaways

- Positional encoding gives position information to the model.
- Sinusoidal encoding uses sine and cosine functions.
- It is added to token embeddings.
- It helps the model understand the sequence order.
- Different variants exist (learnable, relative, rotary, etc.).

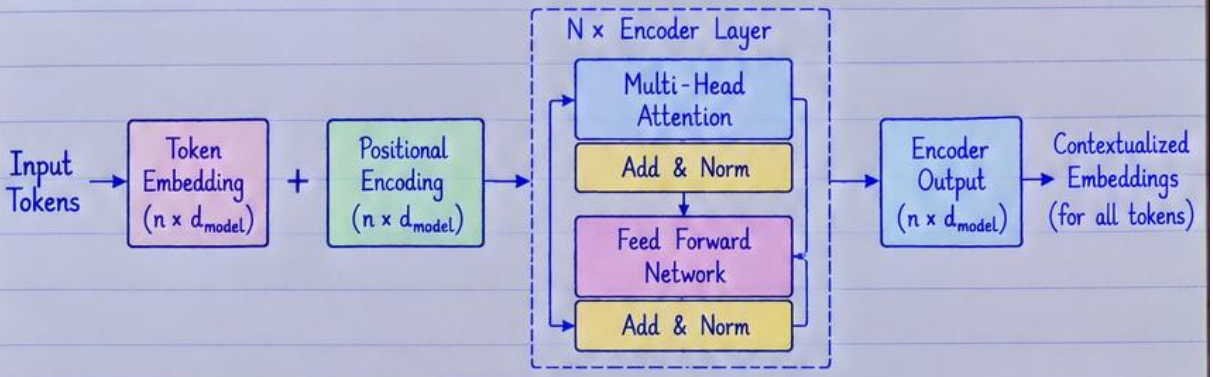
Rahul Yadav

8. Encoder Architecture

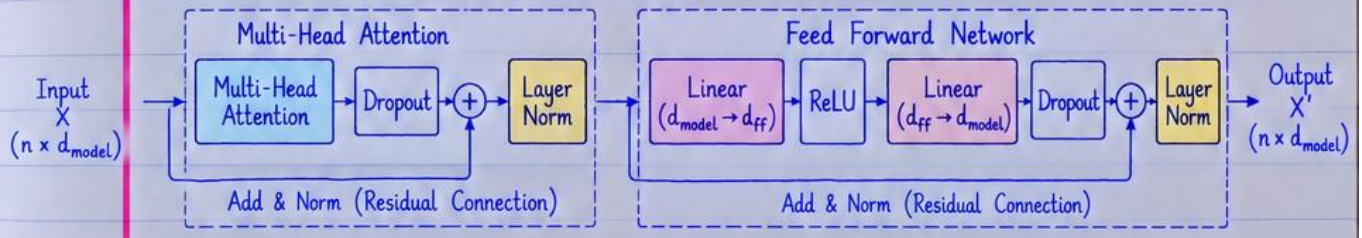
1. What is the Encoder ?

- The encoder is the first part of the Transformer.
- It takes token embeddings with positional encodings as input.
- It consists of a stack of identical encoder layers (N times).
- Each encoder layer contains Multi-Head Attention and a Feed Forward Network.
- It produces contextualized embeddings (also called hidden states) for each token.
- These embeddings capture the meaning of each token based on the entire input sequence.

2. High Level Encoder Architecture

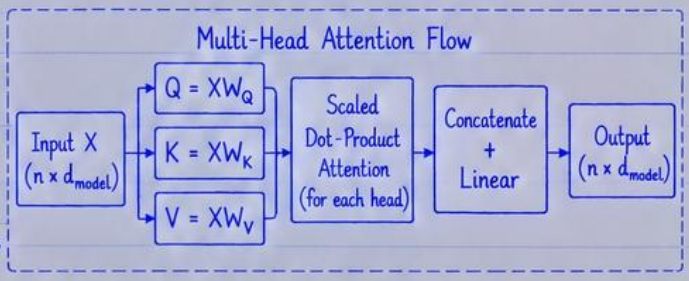


3. Structure of an Encoder Layer



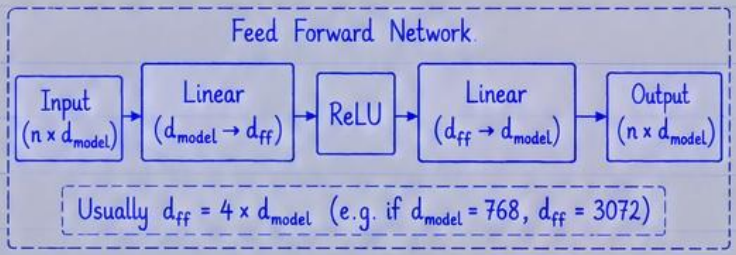
4. Multi-Head Attention in Encoder

- Input X is projected into Queries (Q), Keys (K) and Values (V) using linear layers.
- Multiple attention heads run in parallel.
- Outputs of all heads are concatenated and projected back to d_{model}.
- Helps the model capture different types of relationships between tokens.



5. Feed Forward Network (FFN)

- Two linear transformations with ReLU activation.
- Expands the dimension to d_{ff} (usually $4 \times d_{model}$) and then projects back to d_{model}.
- Applied to each position independently (same weights for all positions).



6. Key Takeaways

- Encoder converts input tokens into rich contextual embeddings.
- Each encoder layer has Multi-Head Attention + Feed Forward Network.
- Residual connections and Layer Normalization help in stable training.
- The same layer is stacked N times (e.g. 6, 12, 24 layers).
- Used in BERT, RoBERTa, etc. (encoder-only models).

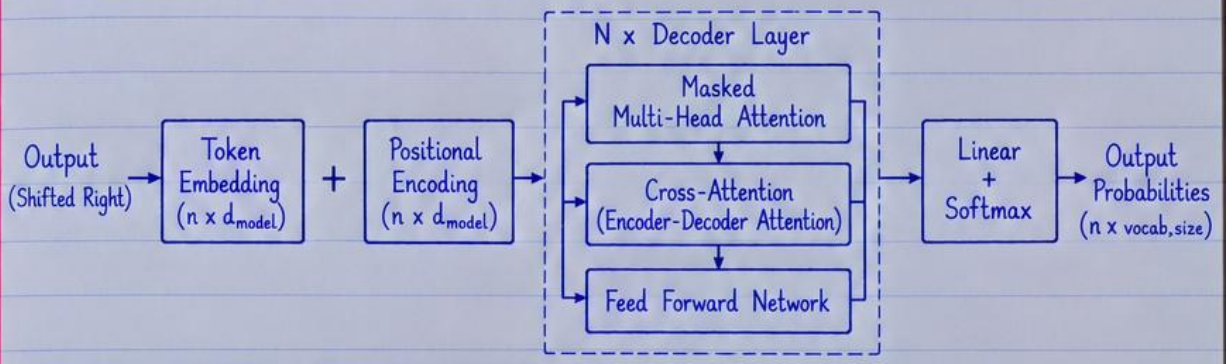
Rahul Yadav

9. Decoder Architecture

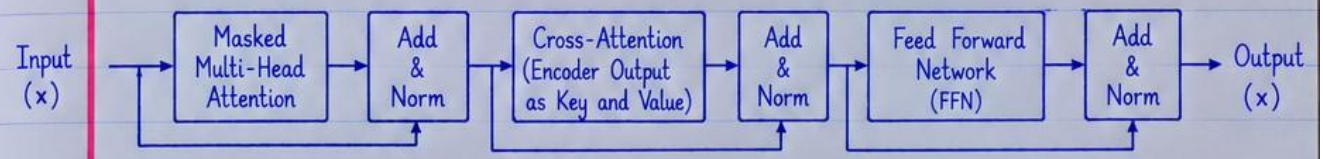
1. What is the Decoder ?

- The decoder is the second part of the Transformer.
- It is used in sequence generation tasks (e.g. machine translation, text generation).
- It generates the output tokens one by one.
- It uses Masked Multi-Head Attention, Cross-Attention and a Feed Forward Network.
- It also uses positional encodings for input tokens (previously generated tokens).

2. Decoder Architecture Overview



3. Structure of a Decoder Layer



4. Masked Multi-Head Attention

- Uses a causal mask (look-ahead mask) to prevent the model from attending to future tokens.
- Each position can only attend to previous and current positions.
- It is the same as self-attention, but with a mask applied.

Causal Mask (for sequence length = 4)

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

← 1 = Attend
0 = Mask (No Attention)

5. Cross-Attention (Encoder-Decoder Attention)

- Queries (Q) come from the decoder (current target tokens).
- Keys (K) and Values (V) come from the encoder output (source tokens).
- Helps the decoder attend to the relevant parts of the input sequence.
- It allows the model to use information from the encoder while generating the output.

6. Feed Forward Network (FFN)

- Same as in the encoder.
- Two linear layers with a non-linear activation (ReLU).
- Expands dimension from d_{model} to d_{ff} and then projects back to d_{model}.
- Applied independently to each position.

FFN Formula

$$FFN(x) = W_2(ReLU(W_1 x + b_1)) + b_2$$

- W_1 : $(d_{model} \times d_{ff})$
- W_2 : $(d_{ff} \times d_{model})$
- Commonly, $d_{ff} = 4 \times d_{model}$

7. Key Takeaways

- The decoder generates output tokens using previously generated tokens.
- It uses masked self-attention, cross-attention and feed forward network.
- Causal masking prevents the model from looking at future tokens.
- Cross-attention helps in using information from the encoder.
- Used in sequence-to-sequence tasks like translation, summarization, etc.

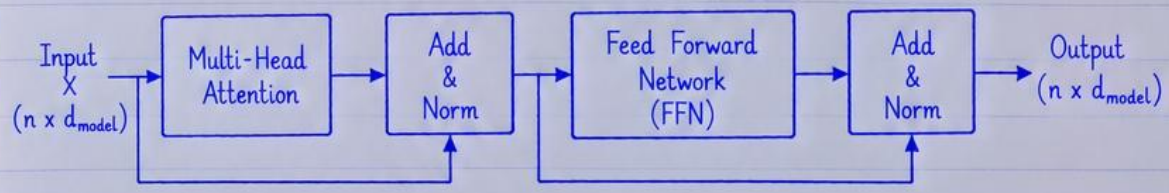
Rahul Yadav

10. Transformer Layer (End-to-End Flow)

1. What is a Transformer Layer ?

- A transformer layer is a combination of Multi-Head Attention, Add & Norm and a Feed Forward Network.
- It processes the input token representations and returns new contextualized representations.
- Multiple such layers are stacked to build the full model.
- Each layer has residual connections and layer normalization to stabilize training.

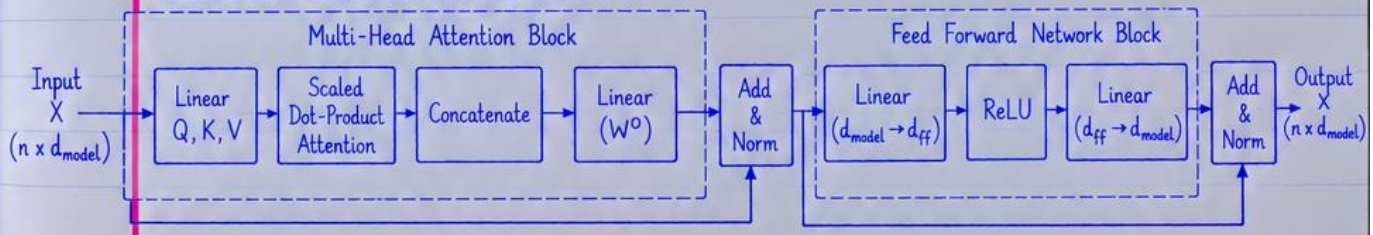
2. Overall Structure of a Transformer Layer



3. Step by Step Flow

1. Input embeddings with positional encodings are passed to the Multi-Head Attention.
2. The output of attention is added to the input (residual connection) and then normalized.
3. The result is passed to a Feed Forward Network.
4. Again, a residual connection and layer normalization is applied.
5. The final output is passed to the next transformer layer (or to the output layers).

4. Detailed Block Diagram



5. Mathematical Formulation

- Let the input to the layer be X $(n \times d_{model})$.
- Multi-Head Attention output:

$$A = \text{MultiHead}(Q, K, V) = \text{MultiHead}(X, X, X) \quad (n \times d_{model})$$

- Add & Norm:

$$Z_1 = \text{LayerNorm}(X + A) \quad (n \times d_{model})$$

- Feed Forward Network:

$$\text{FFN}(Z_1) = W_2(\text{ReLU}(W_1 Z_1 + b_1)) + b_2 \quad (n \times d_{model})$$

where $W_1 : (d_{model} \times d_{ff})$ $W_2 : (d_{ff} \times d_{model})$

- Final Add & Norm:

$$Z_2 = \text{LayerNorm}(Z_1 + \text{FFN}(Z_1)) \quad (n \times d_{model})$$

Z_2 is the output of the transformer layer.

6. Typical Dimensions

- $d_{model} = 768$ (e.g. BERT base)
- $d_{ff} = 4 \times d_{model} = 3072$
- Number of heads $(h) = 12$
- Each head dimension $d_k = d_v = d_{model}/h$

$$d_k = d_v = \frac{768}{12} = 64$$

- Input / Output shape: $(n \times 768)$

7. Key Takeaways

- Transformer layer = MHA + Add & Norm + FFN + Add & Norm.
- Residual connections help in training deep models.
- Layer normalization stabilizes the learning process.
- The same layer structure is used in both encoder and decoder (with slight differences).
- Multiple layers (e.g. 6, 12, 24) are stacked to build powerful models like BERT, GPT, T5, etc.

Rahul Yadav

11. Training a Transformer Model

1. Overview

- A transformer is trained using large amounts of text data.
- The training process teaches the model to predict the next token in a sequence.
- It uses self-attention, positional encoding and a deep stack of transformer layers.
- The objective is to minimize the difference between the predicted token and the actual next token.

2. Training Objective (Next Token Prediction)

- Given an input sequence of tokens, the model predicts the probability distribution for the next token.
- This is a language modeling task.
- The model is trained to maximize the likelihood of the correct next token (or minimize loss).

Given a sequence:	Model predicts:	Training objective:
$x = (x_1, x_2, \ldots, x_{n-1}, x_n)$ →	$P(x_{n+1} \mid x_1, x_2, \ldots, x_n)$ →	Minimize cross-entropy loss

3. Training Pipeline

Large Text Dataset (e.g. Books, Web, Articles) → Tokenization (e.g. BPE, WordPiece) → Input IDs + Positional Encoding → Transformer Model (Forward Pass) → Compute Loss (Cross-Entropy) → Backpropagation (Compute Gradients) → Update Weights (using Adam)

4. Loss Function

- Transformer models use Cross-Entropy Loss for next token prediction.
- For each position, the model outputs a probability distribution over the vocabulary.
- The loss is calculated using the true next token.

$$L = -\frac{1}{N} \sum_{i=1}^{N} \log P(x_{i+1} \mid x_1, x_2, \ldots, x_i)$$

- N : total number of tokens in the sequence
- x_{i+1} : actual next token
- $P(x_{i+1} \mid \ldots)$: predicted probability

5. Optimizer and Training Settings

- Optimizer: Adam (commonly used)
- Learning Rate: usually with a warmup and decay schedule
- Batch Size: large (e.g. 1024, 2048 tokens)
- Sequence Length: fixed (e.g, 512, 1024, 2048)
- Training Steps: many steps on large datasets
- Regularization: dropout, weight decay
- Gradient Clipping: to avoid exploding gradients

Typical Hyperparameters (Example)

- Learning Rate : 1e-4 (with warmup)
- Batch Size : 2048 tokens
- Sequence Length : 1024
- Optimizer : Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
- Weight Decay : 0.01
- Dropout : 0.1
- Training Steps : Hundreds of thousands

6. Training Data

- Large and diverse text data (books, websites, code, articles, etc.).
- Tokenized using subword tokenization (BPE, WordPiece, etc.).
- Helps the model learn grammar, facts, reasoning and general knowledge.

7. Key Takeaways

- Transformer is trained using next token prediction.
- Uses self-attention, positional encoding and deep layers.
- Optimized using cross-entropy loss and Adam optimizer.
- Requires large datasets and computational resources.
- The same training process is used for models like BERT, GPT, T5, etc. (with different objectives or input formats).

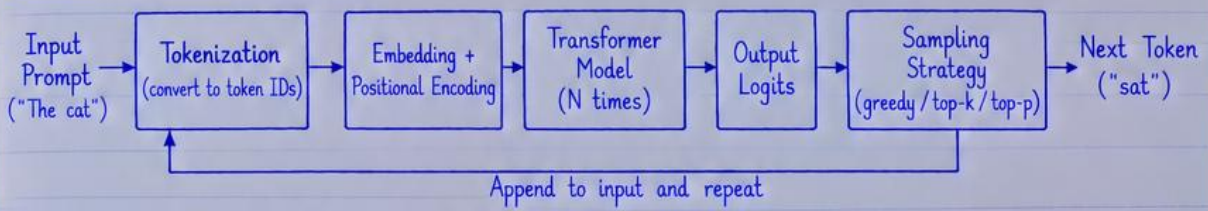
Rahul Yadav

12. Transformer Inference (Text Generation)

1. Overview

- During inference, the transformer generates text one token at a time.
- The model takes the input sequence (prompt) and predicts the next token.
- The predicted token is appended to the input, and the process repeats.
- Different decoding strategies can be used to control the generation (e.g. greedy, beam, top-k, top-p).

2. Inference Flow



3. Step by Step Inference Process

1. Tokenize the input prompt into token IDs.
2. Add positional encoding to the input embeddings.
3. Pass through the transformer layers to get logits for the next token.
4. Apply a decoding strategy to select the next token.
5. Append the selected token to the input sequence.
6. Repeat until a stop condition is met (e.g. max length or end of sentence token).

4. Output Logits and Probabilities

- The model outputs logits (unnormalized scores) for each token in the vocabulary.
- Apply Softmax to convert logits into probabilities.

$$P(w_i) = \frac{e^{\text{logit}_i}}{\sum_{j=1}^{V} e^{\text{logit}_j}}$$

- w_i : token i in vocabulary
- logit_i : model output score for token i
- V : vocabulary size
- $P(w_i)$: probability of token i

5. Decoding Strategies

- **Greedy Search** : Always choose the token with the highest probability.

$$w_{next} = \arg\max_i P(w_i)$$

- **Top-K Sampling** : Choose from top K most probable tokens.
  1. Select top K tokens with highest probabilities.
  2. Sample one token from these K tokens.
- **Top-P (Nucleus) Sampling** : Choose smallest set of tokens whose cumulative probability ≥ p.
  1. Sort tokens by probability.
  2. Select smallest set such that $\sum_{i=1}^{k} P(w_i) \geq p$.
  3. Sample one token from this set.
- **Beam Search** : Keep top B sequences at each step and select the best overall sequence.
  - Helps generate more coherent and high quality text.

6. Stop Conditions

- Generate a special end token (e.g. <eos>).
- Reach maximum sequence length.
- Custom stop sequences (e.g. newline, specific words).

7. Key Takeaways

- Inference generates one token at a time.
- Uses logits and a decoding strategy.
- Different strategies balance between greedy (deterministic) and sampling (creative).
- The same transformer model is used for both training and inference.
- Used in applications like chatbots, summarization, code generation, etc.

Rahul Yadav

13. Efficient Attention & KV Cache

1. Why Efficient Attention ?

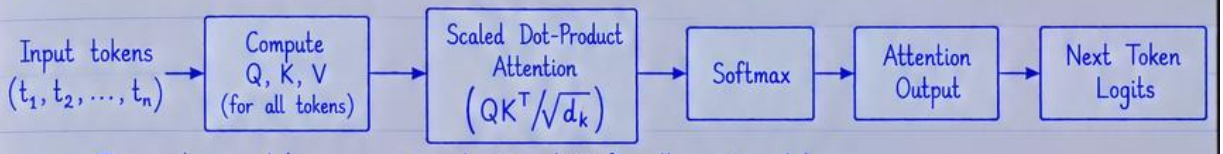
- In standard attention, for each new token we need to recompute attention with all previous tokens.
- This becomes expensive for long sequences because time and memory grow as $O(n^2)$.
- During inference (text generation), we generate tokens one by one, so recomputing attention for all previous tokens is inefficient.
- KV Cache (Key-Value Cache) helps to reuse previously computed Key (K) and Value (V) representations, making inference much faster.

2. Recap: Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q : Query matrix $(n \times d_k)$
- K : Key matrix $(n \times d_k)$
- V : Value matrix $(n \times d_v)$
- d_k : dimension of keys/queries
- n : sequence length

3. Attention Computation During Inference (Without KV Cache)



- For each new token, we recompute K and V for all previous tokens.
- Time Complexity: $O(n^2)$
- Memory Usage: $O(n^2)$

4. KV Cache: Key Idea

- During inference, K and V for previous tokens do not change.
- We compute K and V once and store them in a cache.
- For each new token, we only compute K and V for the new token and concatenate with the cached K and V.

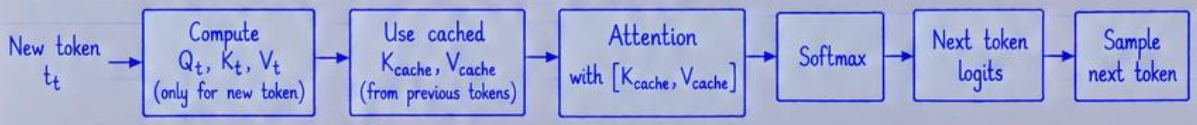
Cache Update at step t

$K_{cache} = [K_1, K_2, \ldots, K_{t-1}]$
$V_{cache} = [V_1, V_2, \ldots, V_{t-1}]$

Compute K_t, V_t for new token

$K_{cache} = [K_1, K_2, \ldots, K_{t-1}, K_t]$
$V_{cache} = [V_1, V_2, \ldots, V_{t-1}, V_t]$

5. Inference with KV Cache (Step by Step)



- Reuse previous K and V from cache.
- Compute attention only with new Q_t and cached K, V.
- Time Complexity per step: $O(n)$ (instead of $O(n^2)$).
- Memory Usage: $O(n \cdot d)$ (linear in sequence length).

6. Benefits of KV Cache

- Much faster inference, especially for long sequences.
- Avoids redundant computation of K and V.
- Used in all modern LLMs (GPT, LLaMA, Mistral, etc.).
- Enables efficient real-time text generation (chatbots, code generation, etc.).

7. Key Takeaways

- Standard attention is expensive ($O(n^2)$) for long sequences.
- KV Cache stores previous Key and Value representations.
- During inference, we compute K and V only for the new token.
- Reduces computation from $O(n^2)$ to $O(n)$ per step.
- Essential for fast and efficient text generation in LLMs.

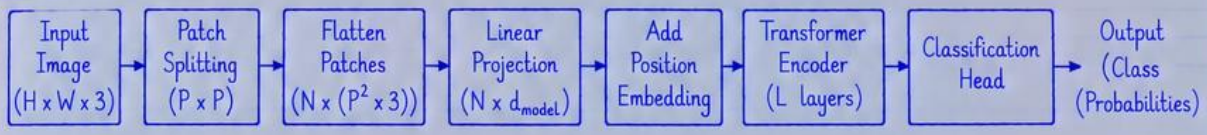
Rahul Yadav

14. Vision Transformers & Transformer Variants

1. Introduction

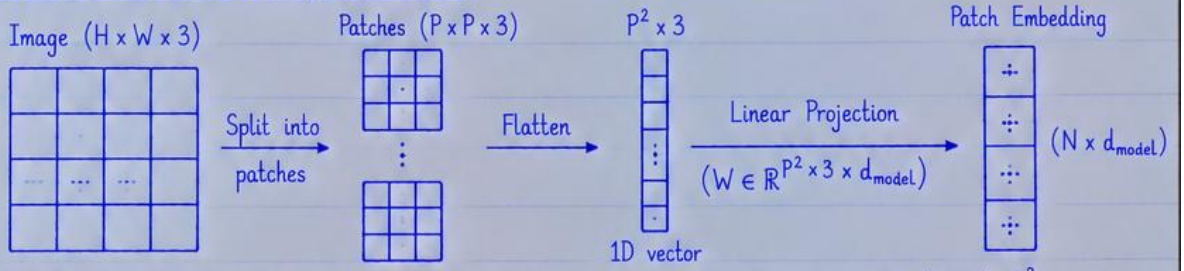
- Vision Transformers (ViT) apply the transformer architecture to image data.
- Instead of words, they use image patches as input tokens.
- They have shown state-of-the-art performance in many computer vision tasks.
- There are many transformer variants designed to improve efficiency, performance and handle different types of data (text, vision, audio, multimodal).

2. Vision Transformer (ViT) Architecture Overview



- The image is divided into small non-overlapping patches.
- Each patch is flattened and projected to a fixed dimension using a linear layer.
- A learnable position embedding is added to keep spatial information.
- The sequence of patch embeddings is passed through standard transformer encoder layers.
- A classification head (e.g. [CLS] token) is used for image classification.

3. Patch Embedding in ViT



- If image size = 224 x 224, patch size = 16 x 16 then number of patches $N = (224/16)^2 = 196$.
- Each patch has size 16 x 16 x 3 = 768, which is flattened and projected to d_{model} (e.g. 768).

4. Position Embedding

- Since transformers are position invariant, we add position embeddings to patch embeddings.
- We can use :
  - Learnable position embeddings (most common).
  - Fixed 2D sine-cosine embeddings.
  - Relative position embeddings (in some variants).
- This helps the model understand the spatial location of each patch in the image.

Final Input to Transformer Encoder

$$Z_0 = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} + \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_N \end{bmatrix}$$

Patch embeddings $(N \times d_{model})$ + Position embeddings $(N \times d_{model})$

5. Popular Transformer Variants

Variant	Key Idea	Used For
BERT	Bidirectional encoder	NLP (language understanding)
GPT	Decoder-only model	NLP (text generation)
T5	Encoder-decoder	NLP (unified framework)
ViT	Transformer for images	Computer Vision
Swin Transformer	Hierarchical windows	Efficient vision model
DeiT	Distillation for ViT	Efficient vision model
BEiT	Masked image modeling	Self-supervised vision
CLIP	Image + Text (multimodal)	Vision-Language Models
BLIP	Vision-language pretraining	Image captioning, VLMs
Perceiver	Cross-attention based	Multimodal and large inputs

6. Key Takeaways

- Vision Transformers treat image patches as tokens and use the same transformer encoder architecture.
- Position embeddings are crucial to maintain spatial information.
- Many transformer variants are designed for different tasks like text, vision, and multimodal learning.
- Variants like Swin, DeiT, CLIP, etc, improve efficiency and enable new capabilities.
- Transformers are a unified and flexible architecture used across multiple domains.

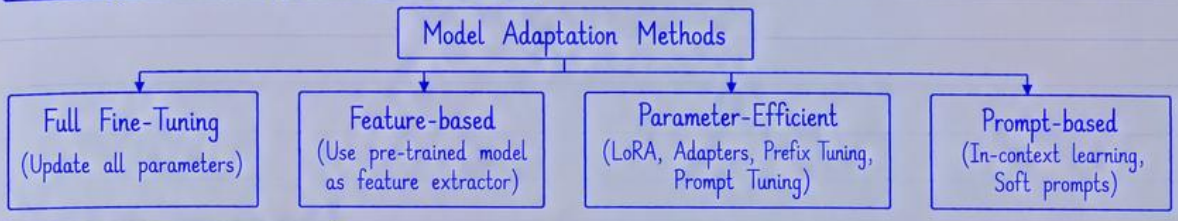
Rahul Yadav

15. Fine-Tuning, LoRA & Adapters

1. Introduction

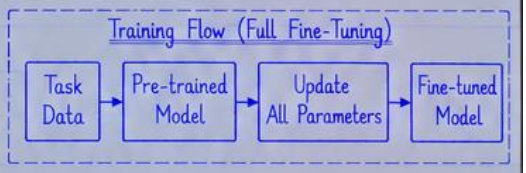
- Pre-trained transformer models (e.g. BERT, GPT, LLaMA) are trained on large general data.
- Fine-tuning adapts the model to a specific task or domain using smaller labeled data.
- However, full fine-tuning is expensive because it updates all model parameters.
- Parameter-efficient techniques like LoRA and Adapters allow us to train only a small number of additional parameters while keeping the original model frozen.

2. Types of Adaptation Methods



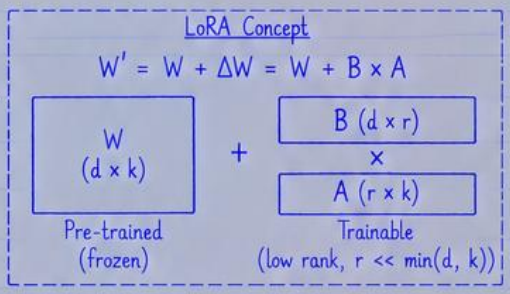
3. Full Fine-Tuning

- All model parameters are updated using task-specific data.
- Requires large computational resources and memory.
- Can achieve the best performance when enough training data is available.
- Risk of overfitting on small datasets.



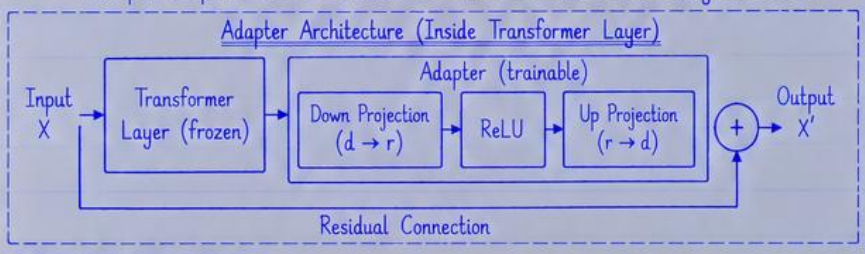
4. LoRA (Low-Rank Adaptation)

- LoRA adds low-rank matrices to the existing weight matrix.
- The original model weights are kept frozen.
- Only the small low-rank matrices are trained.
- Greatly reduces the number of trainable parameters and memory usage.
- Works well for large models like LLaMA, GPT, etc.



5. Adapters

- Adapters are small neural networks inserted into each transformer layer.
- The original model weights are frozen, only adapter parameters are trained.
- Adapters typically use a bottleneck architecture (down-projection and up-projection).
- They are lightweight and can be added to many layers.
- Multiple adapters can be trained for different tasks and easily switched.



Adapter Formula

$$H' = H + W_{up}(\sigma(W_{down}(H)))$$

- H : input hidden states (d)
- W_{down} : ($d \times r$)
- W_{up} : ($r \times d$)
- r : bottleneck dimension (typically 8, 16, 64)
- σ : activation function (e.g. ReLU)

6. Comparison

Method	Trainable Params	Compute Cost	Performance	Use Case
Full Fine-Tuning	All parameters	High	Best (with enough data)	Large dataset, domain specific
Adapters	Small (adapter layers)	Low	Good	Multiple tasks, easy switching
LoRA	Very small (low-rank)	Very low	Good (close to full FT)	Large models (LLaMA, GPT, etc.)
Prompt Tuning	Very small (prompts)	Very low	Moderate	Few-shot / lightweight tasks

7. Key Takeaways

- Fine-tuning adapts pre-trained models to specific tasks.
- Full fine-tuning updates all parameters and is expensive.
- LoRA uses low-rank matrices to efficiently adapt large models.
- Adapters add small trainable modules inside each layer.
- Parameter-efficient methods (LoRA, Adapters, Prefix/Prompt Tuning) enable cost-effective training.
- These techniques make it possible to use large language and vision models for real-world applications.

Rahul Yadav

16. Transformer Optimization, Limitations & Interview Review

1. Training Optimization Techniques

- Transformer models are very large, so several optimization techniques are used to train them efficiently.
- These techniques help in faster training, better convergence and reduced memory usage.
- Common techniques include learning rate scheduling, gradient clipping, mixed precision training, and distributed training.

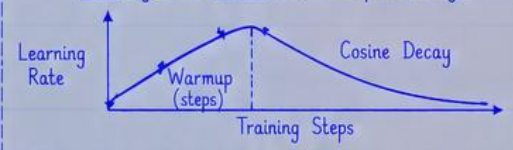
Common Optimization Techniques

- Learning Rate Scheduling (Warmup + Decay)
- AdamW Optimizer
- Gradient Clipping
- Mixed Precision Training (FP16 / BF16)
- Large Batch Training
- Distributed Training (Data / Model Parallel)
- ZeRO Optimization (DeepSpeed)
- Efficient Attention (Flash Attention)

2. Learning Rate Scheduling

- Uses a small learning rate at the beginning (warmup) and then gradually decreases it.
- Helps in stable training and better convergence.
- Widely used schedule: Linear Warmup + Cosine Decay.

Learning Rate Schedule (Warmup + Decay)



3. Mixed Precision Training

- Uses 16-bit (FP16 or BF16) instead of 32-bit (FP32).
- Reduces memory usage and increases training speed.
- Modern GPUs (NVIDIA) support mixed precision with minimal loss in accuracy.

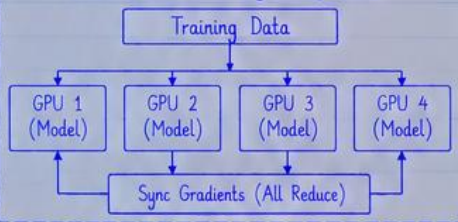
Comparison

Precision	Memory Usage	Training Speed	Accuracy
FP32	High	Slower	High
FP16	Low	Faster	Comparable
BF16	Low	Faster	Comparable

4. Distributed Training

- Large models are trained on multiple GPUs or machines.
- Two main approaches:
  - **Data Parallelism** : Same model on each GPU, different data batches.
  - **Model Parallelism** : Model is split across multiple GPUs.
- ZeRO (Zero Redundancy Optimizer) : reduces memory usage by sharding optimizer states, gradients and parameters.

Data Parallel Training (Simplified)



5. Inference Optimization Techniques

- During inference, we use techniques to make the model faster, smaller and more efficient.
- Common techniques include KV Cache, quantization, model pruning, distillation, and efficient attention.

Inference Optimization Methods

- KV Cache (for fast generation)
- Quantization (INT8, INT4)
- Model Pruning
- Knowledge Distillation (smaller model)
- Efficient Attention (Flash Attention, etc.)
- Batching and Parallel Inference
- Use of optimized runtimes (TensorRT, ONNX, vLLM)

6. Limitations of Transformer Models

- Transformers are powerful, but they also have some limitations.
- They require large amounts of data and computational resources.
- They may struggle with very long contexts and factual accuracy.
- They can generate biased or harmful content.

Key Limitations

- High computational and memory cost ($O(n^2)$)
- Struggle with very long sequences
- Require large training data
- Can hallucinate incorrect information
- May generate biased or toxic content
- Not good at exact arithmetic or logical reasoning
- Lack of real-time knowledge (unless connected to tools)

7. Important Interview Questions (with Short Answers)

1. Why is self-attention better than RNNs? → It can capture long-range dependencies in parallel.
2. What is the time complexity of self-attention? → $O(n^2)$ with sequence length n.
3. What is the purpose of positional encoding? → To add token position information.
4. What is the difference between encoder and decoder? → Encoder understands the input, decoder generates output.
5. What is KV Cache? → It stores computed keys and values for faster inference.
6. What is the difference between LoRA and full fine-tuning? → LoRA trains small low-rank matrices, full fine-tuning updates all parameters.
7. How can we reduce inference latency? → Use KV Cache, quantization, efficient attention, batching, optimized runtimes.
8. What are the main limitations of transformers? → High compute cost, long context issues, hallucinations, bias, etc.
9. What is the use of AdamW? → Optimizer with decoupled weight decay, widely used for transformer training.
10. What is the purpose of attention mask? → To prevent attending to future tokens (in decoder) or padding tokens.

8. Key Takeaways

- Transformers can be optimized for both training and inference using multiple techniques.
- They are powerful but have limitations in compute, context length and factual accuracy.
- Understanding optimization and limitations is important for real-world applications and interviews.

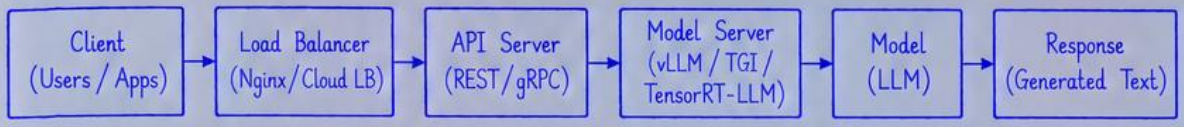
Rahul Yadav

17. LLM Deployment & Serving

1. Introduction

- After training or fine-tuning, the LLM needs to be deployed for real-world usage.
- Deployment involves model optimization, infrastructure setup, and serving techniques to handle multiple users efficiently.
- The goal is to provide low-latency, high-throughput and scalable inference.
- LLMs can be deployed on cloud, on-premise, or edge devices depending on the use case.

2. LLM Serving Architecture

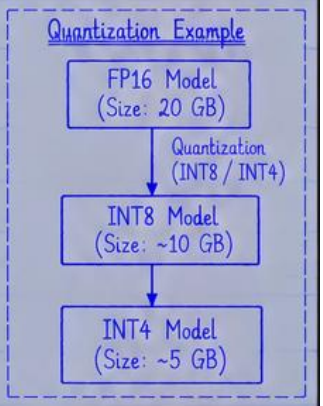


3. Serving Frameworks

Framework	Key Features	Use Case
vLLM	High throughput, PagedAttention, KV cache	Production serving (most popular)
Hugging Face TGI	Easy deployment, supports many models	Quick and scalable serving
TensorRT-LLM	High performance, optimized for NVIDIA GPUs	Low latency, production at scale
Ollama	Run models locally, simple setup	Local development and testing
FastAPI	Build custom APIs for LLMs	Custom applications and services

4. Inference Optimization Techniques

- Quantization (INT8, INT4) : Reduces model size and memory usage.
- KV Cache : Reuse previously computed keys and values for faster generation.
- Batching : Process multiple requests together to improve throughput.
- Flash Attention : Faster attention computation.
- Tensor Parallelism : Split model across multiple GPUs.
- Dynamic Batching : Combine requests arriving at different times.
- Speculative Decoding : Use a smaller model to predict multiple tokens in parallel.
- Model Pruning : Remove unnecessary weights to reduce size.
- Optimized Runtimes : Use TensorRT, ONNX, or vLLM for better performance.



5. Deployment Options

Option	Description	Advantages	Disadvantages
Cloud (AWS, GCP, Azure)	Deploy on cloud GPU instances	Scalable, managed services	Cost can be high
On-Premise	Deploy on own servers	Full control, data privacy	High setup and maintenance
Edge Deployment	Run smaller models on edge devices	Low latency, offline usage	Limited model size
Serverless (e.g. Modal)	Use serverless platforms	Easy deployment, auto-scaling	Cold start latency

6. Real-time vs Batch Inference

Real-time Inference	Batch Inference
Used for interactive applications	Used for offline / large scale tasks
Low latency (few seconds)	Higher throughput
Smaller batch size (1-10 requests)	Larger batch size (100s of requests)
Examples: Chatbot, Q&A, Copilot	Examples: Summarization, data processing, evaluation

7. Monitoring & Scaling

- Monitor latency, throughput, GPU/CPU usage.
- Use tools like Prometheus + Grafana.
- Set up auto-scaling based on traffic.
- Handle model versioning and A/B testing.
- Implement caching for repeated queries.
- Use request queueing to handle high load.
- Ensure security, authentication and rate limiting.

8. Key Takeaways

- LLM deployment requires optimized serving frameworks and infrastructure.
- Techniques like quantization, KV cache, batching and TensorRT help improve performance.
- Choose the right deployment option based on your use case, cost and scalability needs.
- Proper monitoring and scaling are essential for production systems.
- LLM serving enables real-world applications like chatbots, copilots and enterprise AI solutions.

Rahul Yadav

18. RAG (Retrieval Augmented Generation)

1. Introduction

- RAG (Retrieval Augmented Generation) combines the power of LLMs with external knowledge from your own data.
- Instead of relying only on pre-trained knowledge, RAG retrieves relevant information from a knowledge base and uses it in the prompt.
- This helps in generating more accurate, up-to-date and domain-specific responses with citations.

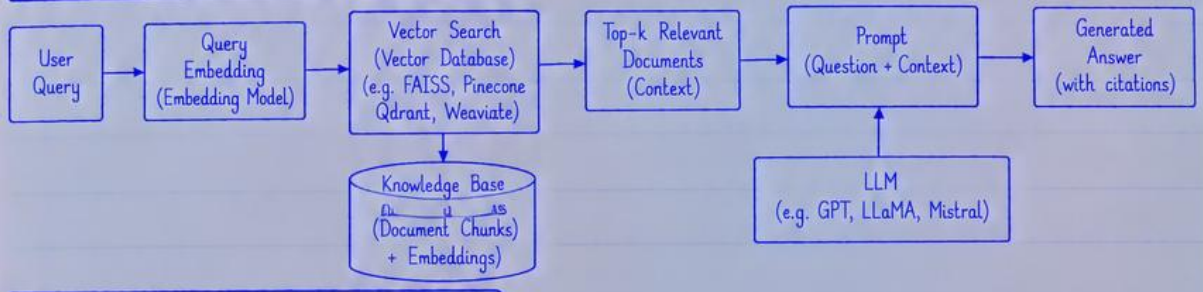
2. Why RAG ?

- LLMs can generate incorrect or outdated information.
- They do not have access to private or domain-specific data.
- RAG allows using your own documents, databases, or websites as a knowledge source.
- Useful for enterprise applications, customer support, research, and internal tools.

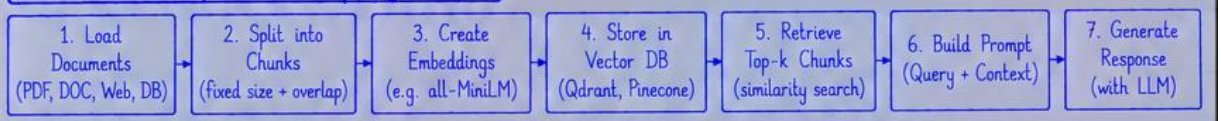
RAG vs Standard LLM

Aspect	Standard LLM	RAG
Data Source	Pre-trained only	Pre-trained + External Data
Accuracy	May be outdated	More accurate
Domain Knowledge	Limited	Can use private data
Citations	Not available	Can provide sources
Use Case	General purpose	Enterprise / Domain specific

3. RAG Architecture (Overview)



4. RAG Pipeline (Step by Step)



5. Document Processing

- Load data from various sources (PDF, text files, websites, databases, APIs).
- Split documents into smaller chunks (e.g. 500 tokens with 50 token overlap).
- Convert each chunk into embeddings using a sentence transformer model.
- Store chunks and embeddings in a vector database along with metadata (source, page number, etc).

6. Chunking Strategies

- Fixed Size Chunking : Split by token size (e.g. 500 tokens).
- Recursive Chunking : Split by headers, paragraphs, sentences.
- Semantic Chunking : Use meaning to create chunks.
- Overlapping Chunks : Add overlap (e.g. 50 tokens) to keep context.
- Metadata : Store source, page number, title, etc.

7. Vector Databases

Database	Type	Key Features	Use Case
FAISS	Open Source	Fast, efficient	Small to medium data
Pinecone	Managed (Cloud)	Scalable, easy to use	Production systems
Qdrant	Open Source	Filtering, payloads	Enterprise use cases
Weaviate	Open Source	Hybrid search, GraphQL	Flexible, scalable

8. Prompt Template (RAG)

```
Use the following context to answer the question.
If the answer is not available in the context, say
"I don't know".
Context:
{retrieved_chunks}

Question: {user_query}

Answer:
```

9. Evaluation Metrics

Metric	Purpose	Description
Relevance	Check retrieved documents	Are the retrieved chunks relevant?
Faithfulness	Check generated answer	Is the answer grounded in context?
Answer Accuracy	Compare with reference	How correct is the final answer?
Latency	System performance	Time taken to generate response
Citation Quality	Source validation	Are the sources useful and correct?

10. Key Takeaways

- RAG improves LLM responses by using external knowledge.
- It involves document processing, embeddings, vector search and prompt augmentation.
- Choose the right chunking strategy and vector database based on your use case.
- Always evaluate the quality, relevance and faithfulness of the generated answers.
- RAG is widely used in enterprise search, chatbots, document Q&A and knowledge management systems.

Rahul Yadav

19. Future Trends, Challenges & Interview Preparation

1. Future Trends in LLMs

- Multimodal LLMs (text, image, audio, video).
- Longer context windows (1M+ tokens).
- More efficient architectures (e.g. Mamba, RWKV, Hybrid models).
- Better reasoning and planning capabilities (Agentic AI).
- Domain specific smaller models for enterprises.
- On-device and edge LLMs for privacy and low latency.
- Integration with tools, APIs and real world data (function calling).
- Improved safety, alignment and controllability.
- Open source ecosystem will continue to grow (e.g. Llama, Mistral).

2. Key Challenges

- High computational and memory requirements.
- Hallucinations and incorrect/factual answers.
- Data privacy and security concerns.
- Bias and fairness issues.
- Keeping models up-to-date with latest information.
- Evaluating and measuring real-world performance.
- Handling long context and complex reasoning.
- Deployment and cost optimization at scale.
- Ethical and responsible AI usage.

3. LLM Evolution (Timeline)

2018 BERT (Encoder) → 2019 GPT-2 (Decoder) → 2020 GPT-3 (Large Scale) → 2022 ChatGPT (Instruction Tuned) → 2023 GPT-4 (Multimodal) → 2024+ Llama, Mistral, Claude, Gemini (Open & Closed) → Future More Capable, Efficient, Safe & Agentic LLMs

4. LLM vs Traditional ML vs Deep Learning

Aspect	Traditional ML	Deep Learning	LLMs (Transformers)
Data Size	Small - Medium	Large	Very Large (Billion+ tokens)
Feature Engineering	Manual	Automatic (learned)	Automatic (from raw text)
Model Type	Classical algorithms	Neural Networks (CNN, RNN)	Transformer (Self-Attention)
Training Time	Minutes - Hours	Hours - Days	Days - Weeks (large compute)
Capabilities	Specific tasks	Vision, Speech, NLP	General purpose, reasoning, multimodal, tool use
Examples	SVM, Random Forest	CNN, LSTM	GPT, Llama, Claude, Gemini

5. Important Concepts to Revise

- Transformer architecture (encoder, decoder).
- Self-attention and positional encoding.
- Pre-training, fine-tuning, instruction tuning,
- Tokenization (BPE, SentencePiece).
- Inference techniques (greedy, top-k, top-p).
- KV cache and efficient inference.
- LoRA, Adapters and model quantization.
- RAG and vector databases.
- Evaluation metrics (Perplexity, BLEU, ROUGE,
- Factuality).
- Deployment and serving (vLLM, TGI, etc).
- Safety, alignment and ethical AI.

6. Sample Interview Questions (with Short Answers)

No.	Question	Short Answer
1	What is self-attention?	It helps the model focus on relevant tokens in the input sequence.
2	Why positional encoding is required?	Transformers are position invariant, so we add positional information.
3	What is the difference between encoder and decoder?	Encoder understands the input, decoder generates output.
4	What is a token?	A small unit of text (word, subword or character).
5	What is KV Cache?	It stores computed keys and values for faster inference.
6	What is LoRA?	A parameter-efficient fine-tuning method using low-rank matrices.
7	What is RAG?	It retrieves external knowledge and uses it to generate better answers.
8	What is the purpose of fine-tuning?	To adapt a pre-trained model to a specific task or domain.
9	What is quantization?	It reduces model size and memory usage (e.g. INT8, INT4).
10	How do you reduce hallucinations?	Use RAG, better prompting, domain data and evaluation guardrails.

7. Best Practices

- Use the right model for your use case (open source or closed).
- Optimize prompts and provide clear instructions.
- Use RAG for domain specific and up-to-date information.
- Evaluate responses using proper metrics.
- Apply quantization and efficient serving for production.
- Implement monitoring, logging and safety guardrails.
- Keep learning and stay updated with the latest research.

8. Final Key Takeaways

- Transformers have revolutionized NLP and AI.
- LLMs are powerful, versatile and continuously evolving.
- Techniques like fine-tuning, RAG, quantization and efficient serving enable real-world applications.
- There are still challenges like hallucinations, bias and cost, but the future is very promising.
- Understanding these concepts is essential for interviews, projects and real-world AI solutions.

"Keep Learning, Keep Building
The Future of AI is in Your Hands !"

Rahul Yadav